

The UI is free; the rules are the law

formal guardrails for LLM agents, act IV: the kernel boundary

Relay — a customer-support SaaS whose UI is hand-written htmx an agent may restyle at will, while every interaction is decided inside a verified kernel API. Guarantees lost to the freedom: zero.

With screenshots of the running product, forged-request tests bouncing off the kernel by rule name, and the honest journal — including the prediction registered before the solver ran. August 2026.

No formal-methods background assumed — the four concepts you need are on slide 3.

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. The bugs that matter live in the interactions you did not think of.
- If we want agents (and background jobs, and time) to act autonomously, we need a way to say what must never break — and have a machine enforce it.

The proposal

The developer writes down intent as rules and invariants. Agents write the rest. Reasoning engines sit between them and arbitrate: every change must provably keep the rules.

Code becomes cheap to regenerate. The spec becomes the durable asset.

Setting: a regular web SaaS application — CRUD/API handlers, authorization, tenancy, workflows, background jobs. Not kernels, crypto, or avionics: every technique here is judged against ordinary product teams, CI budgets, and mainstream languages at the edges.

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “a customer never sees another organization's cases.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves. Here the “model” is not beside the system — the rule base IS both the model and the running program.

Checker

A tireless adversary (here: the Z3 solver). It considers every situation the rules allow — tens of thousands you would never test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact situation that breaks the rule, with the rule that allowed it named. Not “something is wrong” but “an anonymous visitor assigned to a task can start it, via assignee_moves.”

Proofs beside the code: it works — and it has a tax

2 protocols falsified, then proven

P1: the model checker beat a careful engineer twice on an online DB migration. Version 3 survived — and was later proven inductively, for any system size, and live (tracks I, J, M).

59 anomalies per run

P3: skip the guard against real Postgres under concurrent load and the model's predicted anomaly appears 59×/run; 0 with the proven choreography.

1 round to the full fix

P4: same bug, same model, same generic prompt — a stronger frozen gate turned a partial fix into the full fix. Gate strength beats prompt steering.

3.94× faster, never wrong

P5: an agent optimized the CMS freely inside the frozen gate; two correctness-breaking “optimizations” were absorbed mechanically.

Proofs at five levels

Inductive invariants, parameterized (UPDR), liveness under fairness, noninterference via self-composition, a Dafny proof of real session code (tracks I–M).

The residual tax

Every result above polices the model↔code boundary: MBT, trace validation, extraction fidelity. Act II removed that boundary for the ruled part of the system.

Language scouting (notes 11–12): of five ways proofs can meet code, “verified engine + small analyzable DSL” — Cedar's shape — ranked first. Everything after act I builds exactly that.

The rule base is the program; the solver is the reviewer

- One artifact: rules.yaml — roles, lifecycle, allow/deny rules with their ticket sentences. The engine serving it is domain-free; refusals are 403s that NAME the rule.
- One condition grammar, two backends: the analyzer's Z3 view and the server's runtime view are the same parsed rules — agreement checked exhaustively, so spec↔code drift is closed by construction.
- The gate is frozen for agents: safety (EVERY situation) + possibility (SOME — “the safest system does nothing” fails) + feature runs with refusals expected by name + gated lifecycle entries.

```
$ analyze rulesets/cms-buggy --gate rulesets/cms # sabotage
FAIL DEAD allow rule 'editor_read_all' - silently masked
FAIL S2_separation_of_duties counterexample + granting rule
FAIL P2_review_by_non_author: IMPOSSIBLE - strict_privacy
VERDICT: FAIL (7 findings) # 4 detector kinds fire at once
```

Evidence so far (notes 13–14)

- Sabotaged CMS: 7 named findings; naive import extension: 5.
- Background jobs are just actors — a compromised importer is contained by policy, provably.
- Domain transfer: receivables cost the engine ~100 generic lines (time); Flowdeck ~60 (relations).
- Redundant rules proven dead and deleted: the solver shrinks rule bases, reversing 1980s rule rot.

Act III built a full product (Flowdeck) to measure the developer experience. It worked — and it surfaced the tension this deck resolves: who writes the UI?

Flowdeck: the loop is fast and honest — the UI was the tension

1

typo = named load error; the author's dead-rule prediction FALSIFIED by the solver.

2

TWO REAL AUTHORIZATION HOLES (right) + two probes naming the wrong deny.

3

containment deny proven DEAD after the fix — deleted; S11 proves it universally.

4

PASS (0 findings). Tickets → green gate ≈ 1 hour, none of it debugging.

34,560 situations

0.18 s / round

2 real holes

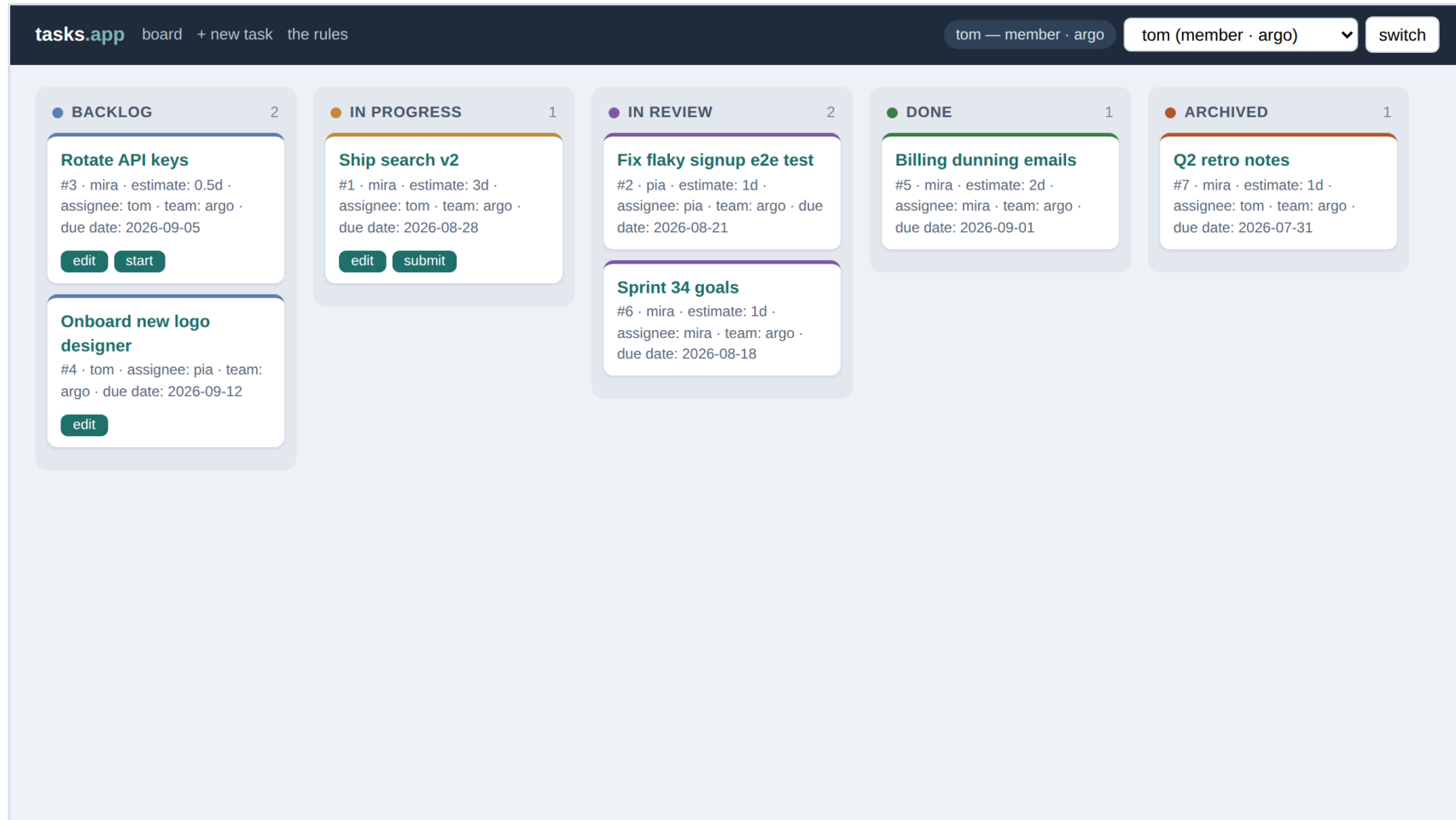
round-2 draft gated forever

```
$ analyze rulesets/taskboard      # round 2, 0.18 s
FAIL S2_no_anonymous_access
  counterexample: {role: anonymous, action: start,
    assigned_to_me: true, ...}
  allowed by: assignee_moves
FAIL S5_assignee_works
  counterexample: {role: admin, action: submit,
    assigned_to_me: false, ...}
  allowed by: admin_oversees
```

Interaction bugs, not typos: each rule read fine alone. Nobody writes the test where a task is assigned to “anonymous”; the $\forall$ -check considers all 34,560 situations and names the granting rule. Full journal: [examples/taskboard/DEVLOG.md](#).

Flowdeck's UI was DERIVED from the rule base — zero app code, boards and buttons for free. Great demo, wrong product surface: a real product's UX is not a reflection of its policy vocabulary. The developer redirected.

Flowdeck's reflected board — now officially scaffolding



Board columns = the lifecycle; cards = the read rule; buttons = the decision function with denying rules named. Everything on screen is a reflection of rules.yaml — which is exactly the limitation. Kept as a diagnostic (engine/ui.py serves any rule base — useful while writing rules). What it can never give a product team: custom queues, brand, workflows, information design. That requires a UI someone OWNS.

Move the boundary down; set the UI free

The developer's requirement (guardrail 10, 2026-08-14): UX customizability is crucial — the implementing agent must be free in the UI, while the interaction logic stays guarded by the rules. The verified boundary moves from “the UI is generated” to a class/function-level kernel API.

FREE — app.py (hand-written htmx, ~600 lines)

Queues, badges, toasts, forms, partial swaps. Agent-authored, restyled at will. Zero policy lives here — so no review of it can ever be about policy.

BOUNDARY — engine/kernel.py (+ the lint that holds it)

visible / get / create / act / edit / delete — every call decided by the rule base BEFORE the store is touched; refusals are typed Denied values naming the rule; decide()/affordances() are pure queries for rendering. App code imports engine.kernel and NOTHING beneath it: `python -m analysis.boundary <app>` fails CI otherwise.

ANALYZED — rulesets/<app>/ (rules + frozen gate)

The program: allow/deny rules from tickets. Z3 reviews every change: dead rules, $\forall$ -safety, $\exists$ -possibility, lifecycle, frozen features with refusals by name. Unchanged from act II — the gate did not get weaker, the surface got freer.

Why not keep the generated UI? It couples the product surface to the policy vocabulary, and it smuggles reflection into the trust story. The UI was never the thing that needed to be correct — enforcement was.

One doorway to state; refusals are values, not prose

```

from engine import kernel  # the app's ONLY engine import

desk = kernel.boot("rulesets/helpdesk", "app.db",
                  today="2026-08-14", seed=...)

rows = desk.visible(actor)      # the read rule, applied
row  = desk.create(actor, {"subject": "Login broken"})
row  = desk.act(actor, "resolve", case_id)
row  = desk.edit(actor, case_id, {"assignee": "sam"})
d    = desk.decide(actor, "close", row)  # pure query

try:
    desk.delete(actor, case_id)
except kernel.Denied as e:
    e.rule.id      # "default_deny"
    e.rule.description # the stakeholder sentence

```

What the boundary guarantees

- Decide, then store — never the other way. A refused mutation writes nothing.
- visible() IS the read rule: there is no unfiltered list to forget to filter.
- affordances(actor, row) returns every action with its decision — buttons, locked-lists, tooltips are one loop; presentation stays the client's business.
- One kernel, three adapters: the JSON API and the old generic UI were refactored onto the same instance. One enforcement path.
- The same 403 vocabulary end to end: ticket → rule id → solver counterexample → Denied.rule → toast.

The kernel is ~200 domain-free lines, written once for every app on the engine. It is small enough to be the thing you verify — the production form is this API over a proven engine (Cedar's shape, note 12).

Note 14's sharpest silent gap — closed at the boundary

Act III's honest friction list had one silent failure mode: rules saw an edit's CURRENT fields, never the proposed values — so “move this case to another org by editing the org field” slipped past the tenancy rule. The kernel now decides every edit twice: may this actor edit this row — and may the row BECOME this?

```
# features.yaml — frozen gate, feat_org_walls
- {actor: dana, action: edit, expect: deny,
  denied_by: org_walls,
  set: {org: zephyr}} # the tenant escape

# rules.yaml — the SAME rule that guards reads
- id: org_walls
  effect: deny
  when: 'actor.role == "customer"
        and not resource.same_org'
```

```
$ pytest tests/ -q # at the kernel, then over HTTP
>>> desk.edit(dana, case_id, {'org': 'zephyr'})
kernel.Denied: denied by org_walls
>>> desk.get(dana, case_id)['org']
'acme' # nothing was written

$ curl -X POST /case/7/edit -d org=zephyr # forged
403 · Refused — rule org_walls
```

- The fix is 6 lines in the kernel + 9 in the pure feature executor (model and implementation must change together — exhaustive backend agreement still holds).
- It applies to every service on the engine at once: Flowdeck's documented “member could re-team a task” hole is retro-closed. All five frozen gates replayed green after the change.
- The ticket sentence became enforceable: “a case can never be moved into or out of an organization by its customers” is now a frozen gate step, expected denied_by org_walls.

Bypassing the kernel is a named CI failure, not a review maybe

Python has no package-private, so the boundary is held the way the frozen gate is held: mechanically, in check.sh. The lint walks the app's AST; the app must pass — and a PRESERVED bypass variant must FAIL, so the lint itself is regression-tested in both directions on every run.

```
# bypass_variant/sneaky_shortcut.py – never run
# a plausible 'quick win': close stale cases in SQL
import sqlite3
from engine import store

def close_stale_cases(db_path):
    conn = sqlite3.connect(db_path)
    for row in store.list_items(conn):
        conn.execute("UPDATE items SET
            state = 'closed' WHERE id = ?", ...)
```

```
$ python -m analysis.boundary app.py
BOUNDARY: ok (imports nothing beneath engine.kernel)

$ python -m analysis.boundary bypass_variant/
FAIL sneaky_shortcut.py:13: imports 'sqlite3':
    the store's substrate is beneath the boundary
FAIL sneaky_shortcut.py:15: imports 'engine.store':
    app code may import engine.kernel only
BOUNDARY: FAIL (2 findings)  # check.sh REQUIRES this
```

- What it refuses: engine internals, sqlite3, reach-around module aliases, mangled kernel attributes. What it skips would not be quiet: three lines of ordinary Python was the whole failure mode.
- Honest scope (note 15): a lint plus name-mangling is not a proof. The by-construction version is a process boundary or a language with visibility (guardrail 8). Falsifier KB1: red-team it.

Relay: a support desk, 7 tickets, round-1 green

```
# TICKETS.md – the product spec (excerpts)
HD-1  Customers see and touch only their own
       organization's cases – reading included.
HD-4  The requester's side decides whether it is
       fixed: only customers reopen. Staff never.
HD-5  Once the SLA due date has passed, an
       ordinary agent may no longer resolve –
       a breached case is resolved by a lead.
HD-7  The mail robot opens cases and files
       replies – and can do nothing else.
```

```
$ analyze rulesets/helpdesk      # round 1, 0.2 s
ok: all 14 rules are effectual
ok   S1_org_isolation ... S13_mailbot_contained
ok   P1_customers_open ... P9_staff_cross_org
ok: 7 transitions live, gated entries respected
ok   feat_lifecycle ... feat_mailbot (42 steps)
VERDICT: PASS (0 findings)
```

- Pre-registered predictions, both held: (a) round-1 PASS; (b) ZERO dead rules — because the five containments (nothing deleted, staff never reopen, only leads close, customers never touch staff states, robot scope) were written with NO deny rules at all. Tight allows + default-deny do the work; S-properties prove each containment universally. Act III's janitor lesson, transferred as method knowledge.
- Honest caveat (DEVLOG): third domain by the same author — round-1-green says the discipline transfers, not that the gate is superfluous. The same author produced two real holes one domain ago; the gate stays frozen for the author who hasn't read the DEVLOGs.
- Engine domain vocabulary added for Relay: 0 lines — org tenancy, assignment and SLA reused act III's projections (same_org, mine, sla_breached). The note-14 transfer-cost prediction confirmed.

Nobody generated this — and nobody reviewed it for policy

The screenshot shows a support desk interface. On the left is a dark sidebar with the following items:

- Relay (selected)
- support desk · rules-guarded
- Inbox (3)
- Working (2)
- On hold (2)
- SLA breached (1)
- Resolved (1)
- Closed (1)
- + New case
- signed in as sam (agent)
- desk date: 2026-08-14

The main view is titled 'Working' and shows '2 case(s) · what sam may see, nothing more'. It contains a table with the following data:

SEV	CASE	ORG	STATE	ASSIGNEE	SLA
● high	Login broken for SSO users	acme	working	sam	due 2026-08-20
● high	API 500s on bulk upload	zephyr	working	sam	due 2026-08-16

The agent-written surface: dark sidebar, queue navigation with live counts, severity dots, org chips, SLA due dates. htmx partial swaps against the kernel. ~600 lines of UI in which no policy can live.

python examples/helpdesk/app.py → http://127.0.0.1:8810/ — personas switchable in the sidebar; the demo desk is seeded THROUGH the kernel as the real personas, so a seed that violates policy cannot exist.

“SLA breached” — a queue no reflected UI could invent

Relay

support desk · rules-guarded

Inbox3

Working2

On hold2

SLA breached1

Resolved1

Closed1

+ New case

signed in as

sam (agent)

desk date: 2026-08-14

SLA breached

1 case(s) · what sam may see, nothing more

SEV	CASE	ORG	STATE	ASSIGNEE	SLA
med	Fwd: cannot reset password	acme	inbox	unassigned	⚠ 2026-08-12 (2d over)

A cross-state product view (new+open+waiting, past due), with red day-counters — pure presentation over `kernel.visible()`, invented by the UI author because support teams triage by breach, not by lifecycle state.

This is what “customizability is crucial” buys: the rule base knows states and dates; the PRODUCT decides what a queue is. The count differs per persona because `visible()` is the read rule.

Same desk, signed in as the customer: one org exists

Relay

support desk · rules-guarded

Inbox

2

Working

1

On hold

1

SLA breached

1

Resolved

1

Closed

1

+ New case

signed in as

dana (customer · acme)

desk date: 2026-08-14

Working

1 case(s) · what dana may see, nothing more

SEV	CASE	ORG	STATE	ASSIGNEE	SLA
high	Login broken for SSO users	acme	working	sam	due 2026-08-20

dana (customer, acme) sees acme's single working case; zephyr's cases and counts are simply absent. org_walls is one deny rule; S1_org_isolation proves no allowed situation crosses it — the sidebar is the proof.

The forged-request tests POST reads and edits on the invisible cases anyway: 403, named org_walls, nothing written (tests/test_relay.py).

Allowed actions are buttons; refused ones tell you why

Relay

support desk · rules-guarded

Inbox

3

Working

2

On hold

2

SLA breached

1

Resolved

1

Closed

1

+ New case

signed in as

quinn (agent)

desk date: 2026-08-14

← queues / case #1

Login broken for SSO users

working

acme

due 2026-08-20

requester

dana

assignee

sam

severity

high

case id

#1

Put on hold

Assign to me

▼ 1 locked action(s) — the kernel refuses these for quinn

Resolve — refused by only_assignee_resolves (press to see)

subject

Login broken for SSO users

assignee

sam

severity

high

SLA due

08/20/2026

org

acme

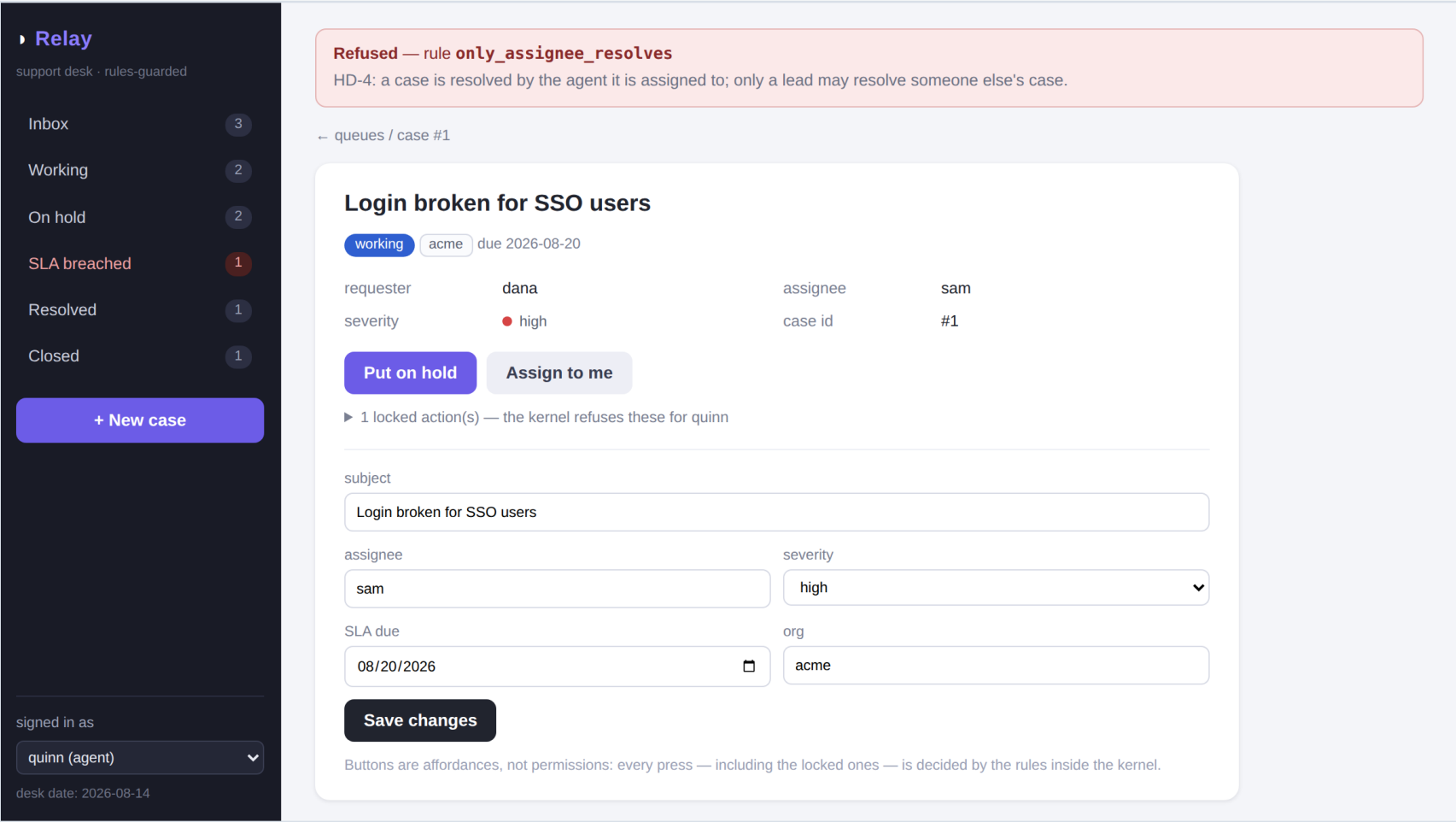
Save changes

Buttons are affordances, not permissions: every press — including the locked ones — is decided by the rules inside the kernel.

quinn (staff, not the assignee) gets Put-on-hold and Assign-to-me live, and one locked action: “Resolve — refused by only_assignee_resolves.” One kernel call (affordances) powers both lists; the presentation philosophy — disclosure vs. greying — is the UI author’s choice.

The locked entries stay pressable on purpose: the UI reflects decisions, it never makes them.

Press the locked button anyway: the ticket sentence answers



403 — Refused: rule `only_assignee_resolves`, “HD-4: a case is resolved by the agent it is assigned to; only a lead may resolve someone else’s case.” Ticket → rule id → solver vocabulary → typed Denied → toast: one unbroken chain of names.

htmx quirk found here: 4xx responses are not swapped by default — the honest 403 needed a one-listener opt-in. A free-layer bug, caught by a screenshot, with nothing at stake but pixels (DEVLOG).

Nothing — and the tests prove it, by forging what the UI hides

```
# tests/test_relay.py — the UI renders postbot no
# buttons at all; forge the POST anyway:
fetch(port, "POST", f"/case/{id}/act", "postbot",
      body="action=triage")
assert status == 403 and 'default_deny' in text

# quinn is staff, but not the assignee:
fetch(..., "quinn", body="action=resolve")
assert 'only_assignee_resolves' in text
assert state_unchanged

# dana forges the tenant escape over HTTP:
fetch(..., "dana", body="org=zephyr")
assert 'org_walls' in text and org == 'acme'
```

The asymmetry, measured

- Hiding a button changes nothing: every press and every forged request is decided inside the kernel.
- The seed had to obey the rules too: seed case 6 breaches its SLA, so the seed script COULD NOT resolve it as the agent — it routes through the lead.
- The free layer's own bugs (hx-vals quoting, the 4xx swap) burned plumbing time and zero policy risk. That asymmetry is the architecture working.
- 12 tests at three layers: model agreement (19,200 situations), kernel refusals by name, app over HTTP.

Kept honest — each one is a falsifier on the table

default_deny flattens refusal UX

Omitting provably-dead containment denies is right by the analyzer — but the robot's refusal now reads “no rule allows this” instead of a stakeholder sentence. Open idea (KB4): explain refusals by the nearest allow — “mailbot_files grants open, reply — not triage.” An explanation is a disclosure channel; org walls must apply to error messages too.

The lint is conservative, not complete

It blocklists imports and reach-around attribute names. KB1: red-team it — if a plausible (not adversarial) bypass passes, the boundary needs AST call analysis or a process split.

Two decisions per edit ≠ transactions

A concurrent writer between decide and write is unhandled (single SQLite connection hides it). KB3: the P3 harness pattern — real Postgres, concurrent load — applied to the kernel.

Reads may outgrow the kernel

Aggregations, search, pagination will pressure “just open the DB read-only.” KB2: a reporting seam that still applies the read rule per row is the test of reads-as-decisions at scale.

One entity per rule base

Relay's cases fit one rule base; case comments/attachments would force relations between ruled entities — N rule bases + client joins, or engine work.

The engine is unproven Python

~1,900 domain-free lines taken on faith. The production form: this kernel API over a verified engine (Cedar's Lean proofs, or the track-D Dafny→Go shape) — one proof, then per-change analysis only.

Five rule bases, one engine, every claim executable

5 : 1

services per engine — CMS, tickets, receivables, Flowdeck, Relay on ~1,900 lines of domain-free Python (kernel: 209; boundary lint: 105).

0 lines

of domain vocabulary the engine needed for Relay — the transfer trend closed: time (~100) → relations (~60) → zero, as predicted in note 14.

89,856

situations checked exhaustively across the five services — runtime evaluation and the Z3 compilation agree on every one.

2-way CI

check.sh holds both directions: Relay's gate must PASS, the preserved bypass variant must FAIL the lint, the taskboard's round-2 draft must FAIL the frozen gate.

Relay by the numbers — the whole product:

- 361 lines of YAML (14 rules + 13 $\forall$ -properties + 9 $\exists$ -witnesses + 2 gated lifecycle entries + 42 frozen feature steps) — reviewed by the solver in ~0.2 s per change.
- ~600 lines of free htmx UI (app.py) — reviewed by nobody, for policy, ever: the boundary lint proves it cannot touch state around the kernel.
- 12 tests across three layers; 5-stage check.sh; screenshots reproducible (screenshots.py); journal in DEVLOG.md.

What the analyzer asks of every rule change, in seconds: dead rules · stale assumptions · $\forall$ -safety · $\exists$ -possibility · lifecycle liveness + gated entries · frozen features with refusals by name.

Learned the hard way — the newest one is this deck's thesis

10 · The boundary is a kernel API, not a UI

NEW. Rules guard interaction logic behind a function-level API; the UI above is agent-authored and free; generated UIs are scaffolding. Hold the boundary mechanically (lint), never by convention.

2 · Gates need both directions

Safety-only gates accept fixes that trade away liveness (“the safest system does nothing”). Every gate = safety + possibility witnesses + frozen feature runs.

6 · Counterexamples are the currency

Named and machine-readable: situation witnesses, unsat cores, denied_by 403s — now typed Denied values any UI renders its own way.

8 · Boundaries by construction

Where the language allows: capability tokens, tystate. Where it doesn't (Python): name-mangling + a lint whose failure is named — and the honesty that this is not a proof.

1 · The spec is frozen for agents

The analyzer takes --gate from a pinned directory; agents edit rules, never the gate. Historic buggy drafts are gated forever as regressions.

3 · Gate strength beats NL steering

Same bug, model, prompt: a stronger gate turned a partial fix into the full fix (P4). Cheaper models are fine when the gate is strong.

7 · Record falsified predictions

The DEVLOGs keep wrong predictions next to the runs that killed them (act III) and pre-registered predictions next to the runs that confirmed them (act IV).

5 · Translate / validate split

LLM translation of NL→rules is unsound — always followed by a sound solver step. Humans review rules sentence-by-sentence, never agent edits.

Falsifiers on the table

Held so far

- RB1 (tickets land as rule diffs): held four times, and the cost curve closed — imports 0, receivables ~100 (time), Flowdeck ~60 (relations), Relay 0. Note 14's zero-lines prediction confirmed.
- DX verdict (notes 14–15): the loop beats conventional development for the policy-shaped core, and the redirect resolved its biggest tension — product UX is now free without weakening a single gate.

Next

- KB1 — red-team the boundary lint: a plausible bypass that passes forces AST call analysis or a process split. KB2 — a reporting/read seam (search, aggregation) that still applies the read rule per row.
- KB3 — the P3 harness against the kernel: concurrent writers between decide and write, on real Postgres. KB4 — nearest-allow refusal explanations that do not leak across tenants.
- DX1 — an outside developer ships a ruled app (rules + free htmx UI) from tickets in under a day. DX2 — the same tickets implemented conventionally by a strong LLM: does it contain the interaction holes the gates caught?
- RB3 — ticket→rule-diff vs ticket→handler fidelity, measured. RB4 — the Quint temporal twin (races). P8 — the same rules as Cedar policies. P9 — the invariant→Postgres-constraint compiler nobody has built.

Parked (consciously, with reasons recorded)

Alloy (Z3 covers it) · Quint→Rust codegen (no tooling) · Kani until unbounded domains arrive · Verus (proxy-blocked; Dafny stands in).

The one-sentence takeaway

Split the app at a kernel API: below it, 361 lines of solver-reviewed rules that hold the product to its tickets in every one of 19,200 situations; above it, a hand-written UI an agent may redesign at will — because the tests forge every request the UI hides, and the kernel refuses each one by name.

Run it: `examples/helpdesk/check.sh` · `python examples/helpdesk/app.py` → `http://127.0.0.1:8810/`

Read it: `examples/helpdesk/DEVLOG.md` (the journal) · `research/15-kernel-boundary-free-ui.md`

The stack beneath: notes 13–14 (rules as the program, the DX study) · 09–12 (proof escalation) · p1–p7